

Сети: от TCP до TLS

Реферат по сессиям 09-12

30 сентября 2026 • Модуль 02 • Учебный homelab

Четыре задачи, одна модель: сначала понять, на каком участке оборвался запрос, затем изменить только доказанную причину. Материал предназначен для повторения и коротких записей от руки.

Карта прочитанного

- 1. Путь запроса и границы доказательств.
- 2. Session 09: connection refused и timeout.
- 3. Session 10: проху отвечает HTTP 502.
- 4. Session 11: HTTP 200, но версия не та.
- 5. Session 12: действующий сертификат для другого имени.
- 6-7. Команды по темам и классам действий.
- 8. Выжимка, самопроверка и источники.

Общая модель

имя -> адрес -> TCP -> TLS (для HTTPS) -> HTTP -> данные
проху -> отдельное соединение к API

У каждого слоя свой вопрос. Resolver выбирает адрес. TCP устанавливает соединение с адресом и портом. TLS защищает соединение и позволяет проверить сервер. HTTP описывает запрос и ответ. Прикладной контракт определяет, какие данные мы должны получить.

Что на самом деле доказывает проверка

- active: systemd видит работающий процесс, но не гарантирует доступность URL.
- LISTEN: существует принимающий socket; маршрут клиента может быть неверным или заблокированным.
- HTTP 200: конкретный запрос обработан успешно по HTTP, но версия и содержимое могут быть неправильными.
- Успешный исходный URL с нужным телом и включёнными TLS-проверками: более сильное доказательство клиентского результата.

Статусы без приукрашивания

09: практика восстановлена с помощью; устная защита остаётся незавершённой. 10-12: зачтены с подсказками и уточнениями на защите. Все PASS этих чекеров сообщены мной; повторный полный запуск преподавателем не выполнялся. В 12 есть скриншот успешного HTTPS с правильным JSON.

02 / Два разных отказа TCP

Session 09 • 24-27 сентября

Исходная ситуация

Запрос к 127.0.0.1:8120 завершался немедленно ошибкой curl (7). Запрос к 127.0.0.1:8121 ждал около трёх секунд и завершался curl (28). HTTP-кодов не было.

Диагностика и причина

У s9-a состояние inactive, на 8120 нет listener. После запуска службы endpoint восстановился. У s9-b процесс active, на 8121 есть listener, но это ещё не исключало сетевую фильтрацию. После разбора nftables выявлен DROP в отдельной учебной таблице inet s9_lab.

Таблица объединяет цепочки; цепочка содержит правила. Условия правила выбирают пакет, действие определяет, что с ним сделать. DROP отбрасывает пакет без ответа. В этой задаче это объясняло таймаут при живом приложении.

Что я исправил

- А: я запустил только s9-a.
- В: я удалил только изолированную учебную таблицу s9_lab. Это шире удаления одного правила, но не сброс всего firewall.
- Я не отключал UFW, не очищал чужие таблицы и не перезапускал исправный s9-b.

Почему это не универсальный диагноз

Refused часто указывает на отсутствие listener или активное отклонение. Timeout означает отсутствие результата за отведённое время, но сам по себе не доказывает DROP: возможны проблемы маршрута, потери или ожидание приложения. curl (28) может возникнуть не только на этапе TCP. Нужны дополнительные наблюдения.

Проверка и границы evidence

После удаления фильтра на скриншоте: HTTP 200 и JSON service=s9-b. Я сообщил все PASS. Повторная проверка 27 сентября подтвердила active, endpoints, ответы и текущие журналы, но не повторила root-only checker и отдельно не проверила отсутствие таблицы и владение сокетами по PID.

Ошибка, которую исправили в понимании

Чтение огромного ruleset не должно быть угадыванием. Сначала определяем адрес, порт и точку отказа; затем ищем относящиеся к ним условия и счётчики. Имена чужих таблиц не являются поводом для их удаления.

В блокнот

Нет HTTP-кода: сначала проверяю этап соединения. Процесс, listener и путь пакета - разные уровни. Живой процесс не отменяет firewall.

03 / Два участка запроса

Session 10 • HTTP 502 • 28 сентября

Исходная ситуация

Клиент обращался к проху на 127.0.0.1:8122/health и получал HTTP 502, service=s10-proxu, error=bad_gateway. Следовательно, до проху запрос дошёл. Отказ требовалось искать дальше, а не объявлять недоступной всю сеть.

```
клиент -> проху :8122 -> UPSTREAM :8124 (нет listener)
API реально слушает :8123
```

Ход расследования

В конфиге найден UPSTREAM на 8124. Я предложил проверить listener и сообщил, что он отсутствует. После подсказки по поиску s10-api и EnvironmentFile установлено: API работает на 8123. Не «порт не существует», а «на этом адресе и порту никто не слушает».

Ранний успешный curl к 8121 попадал в прошлую лабораторную s9-b. Успех другого сервиса не проверяет нужный тикет. Перед командой важно сверять исходный URL.

Минимальное исправление

Я поменял upstream-порт 8124 на 8123 и перезапустил только s10-proxu. API уже работал правильно. Код приложения и unit не менялись.

EnvironmentFile перечитывается при запуске нового процесса службы. В этом случае нужен restart, а не daemon-reload. Если меняется сам unit или drop-in, systemd должен перечитать определения через daemon-reload; это само по себе не перезапускает приложение.

Приёмка

Проверять нужно URL на 8122 через проху: HTTP-код и ожидаемое тело. Прямой API-ответ полезен для локализации, но не заменяет этот тест. О восстановлении и всех PASS я сообщил в чате; отдельного финального curl-скриншота этой сессии нет.

Уточнение на защите

HTTP 200 - не TCP-рукопожатие. HTTP-ответ приходит после установления соединения и относится к конкретному запросу. Исправный API при проху 502 не означает исправный клиентский путь.

В блокнот

502 здесь: проху достигнут, backend недоступен по настроенному адресу. Сверяю UPSTREAM с фактическим listener, меняю одну причину и повторяю исходный запрос.

04 / Успешный код, неверные данные

Session 11 • HTTP-контракт • 29 сентября

Исходная ситуация

Три службы active. Proxy на 8130 направляет запросы на old/v1 на 8132; нужная api/v2 слушает 8131. Старый backend отвечает HTTP 200, поэтому только по коду ответа дефект не обнаружить.

Самостоятельная часть

Я сопоставил status и журналы трёх служб, определил неправильную версию backend и предложил переключить upstream. Это самостоятельное применение модели предыдущего инцидента.

Исправление и наблюдение

Я изменил upstream в /etc/s11-proxy.env и перезапустил только proxy. Скриншот показывает новый MainPID 212937 и upstream=http://127.0.0.1:8131. Это подтверждает применение настройки, но не заменяет проверку клиентских маршрутов.

Приёмка по контракту

Для /health и /items проверяется не только код, но и нужная версия, идентичность сервиса и ожидаемые данные. Команды приёмки подсказаны; все PASS сообщены мной. Финальные ответы отдельно не приложены.

Ошибки и полезные ограничения

- Случайно открыт конфиг прошлой лабораторной. Сохранение и восстановление той правки не подтверждены; не делаем вид, что всё автоматически откатилось.
- Удаление старой версии не является первым шагом. Сначала переключаем потребителей и проверяем результат; затем подтверждаем, что старый сервис никому не нужен.
- Проверка порта не обнаружит правильный HTTP-ответ с неправильными данными.

Что такое контракт в этой практике

Ожидания клиента: адрес и маршрут, допустимый код, структура ответа и значения существенных полей. Исправность транспорта - необходимое условие, но не гарантия выполнения прикладной задачи.

В блокнот

HTTP 200 может прийти от неправильного приложения. Проверяю не просто «кто-то ответил», а «нужный сервис вернул нужные данные через нужный маршрут».

05 / Сертификат для другого имени

Session 12 • HTTPS • 29-30 сентября

Исходная ситуация и проверка

В конфиге s12-api указаны old.crt и old.key. Название old не доказывает просрочку. openssl показал срок 29 сентября - 29 октября 2026 и SAN DNS:old.s12.test. Клиент же использует api.s12.test: проблема в имени.

Затем я проверил ca.crt - это сертификат центра сертификации, а не удостоверение имени API. Для выбора серверного сертификата я проверил api.crt: его SAN содержит DNS:api.s12.test, срок действующий.

Три разные сущности

- Серверный сертификат: публичный ключ, имена и срок; подписан CA.
- Приватный ключ сервера: нужен серверу, должен соответствовать сертификату; не публикуется и не отправляется в чат.
- Сертификат CA: клиент использует его как доверенную основу для проверки цепочки.

Имя файла не входит в проверку hostname. Переименование old.crt ничего не исправляет.

Редактирование SAN внутри подписанных данных нарушает подпись. Нужен сертификат, выданный для требуемого имени, и соответствующий ключ.

Изменение и проверка

По подсказке я выбрал /etc/s12/api.crt и /etc/s12/api.key в env-файле и перезапустил s12-api. На финальном скриншоте исходный HTTPS-запрос с --cacert и без -k возвращает HTTP 200 и JSON status=ok, service=s12-api. Все PASS checker сообщены мной.

```
curl -q --noproxy '*' -i --max-time 4 \  
  --cacert /etc/s12/ca.crt \  
  --resolve api.s12.test:8443:127.0.0.1 \  
  https://api.s12.test:8443/health
```

--resolve задаёт адрес подключения для пары имя:порт, сохраняя имя URL для TLS. --cacert задаёт доверенный CA для запроса. -k отключил бы проверку подлинности: шифрование осталось бы, но защита от подмены сервера ослабла. Это не исправление сертификата и не настройка браузеров.

В блокнот

Действующий срок не гарантирует нужное имя. Не отключаю проверку ради 200. Проверяю исходный HTTPS URL, доверие, имя и тело ответа.

06 / Команды: службы и сеть

Шпаргалка по темам и классам

Метки: ПРОСМОТР - без намеренного изменения конфигурации; ПРОБА - запрос может оставить лог; ИЗМЕНЕНИЕ - меняет состояние; УДАЛЕНИЕ - требует точной области. Шаблоны с <...> не копируются буквально.

systemd / состояние и конфигурация / ПРОСМОТР

```
systemctl status <unit>
systemctl cat <unit>
journalctl -u <unit> -n 30 --no-pager
```

status: Active, MainPID и последние сообщения. cat: определение unit и drop-in, включая EnvironmentFile. journalctl: -u выбирает службу, -n ограничивает число записей, --no-pager выводит без просмотрщика. Пример службы: s10-проху. Старые строки журнала не доказывают состояние нового процесса; сверяй время и PID.

systemd / жизненный цикл / ИЗМЕНЕНИЕ

```
sudo systemctl start <unit>
sudo systemctl stop <unit>
sudo systemctl restart <unit>
sudo systemctl daemon-reload
```

start запускает, stop останавливает, restart создаёт новый запуск. daemon-reload перечитывает определения systemd, не заменяя restart приложения. В s10/s11 я менял env и перезапускал только проху. Reboot всего сервера для этого не нужен.

Сеть / listener и разрешение имён / ПРОСМОТР

```
sudo ss -ltnp 'sport = :8123'
getent ahostsv4 api.s8.test
grep 'api.s8.test' /etc/hosts
```

ss: -l слушающие, -t TCP, -n числовые адреса, -р процессы; sport фильтрует локальный порт. Смотри Local Address:Port и PID. getent показывает системное разрешение IPv4, а grep ищет локальную запись. Эти последние две команды повторяют Session 08, не являются новой практикой 09-12.

Firewall / исследование и точечное изменение

```
sudo nft list ruleset
sudo nft delete table inet s9_lab
```

Первая - ПРОСМОТР, вторая - УДАЛЕНИЕ из исторического решения s9. Не запускать повторно «для проверки». В обычной системе нельзя удалять таблицу по похожему названию: сначала нужны содержимое, владельцы правил и согласованная область изменений. Полный flush ruleset я не применял.

07 / Команды: HTTP, TLS и Linux

Продолжение тематической шпаргалки

HTTP / клиентская проба / ПРОБА

```
curl -i --max-time 3 http://127.0.0.1:8122/health
```

-i добавляет заголовки к телу; --max-time ограничивает весь запрос секундами. Смотри ошибку соединения или HTTP-код, затем JSON. Не подменяй URL проху прямым URL API при финальной приёмке.

TLS / публичный сертификат / ПРОСМОТР

```
openssl x509 -in /etc/s12/api.crt \  
-noout -dates -ext subjectAltName
```

x509 читает сертификат; -in выбирает файл; -noout убирает печать всего PEM; -dates показывает срок; -ext выбирает SAN. Сравни DNS-имя с URL. Команда не проверяет наличие соответствующего приватного ключа на сервере и не заменяет HTTPS-пробу.

Для полного curl из Session 12: -q первым аргументом отключает чтение пользовательского curl-конфига; --проху "*" исключает проху-переменные; --resolve задаёт адрес, --cacert доверие. Пример приведён на странице 5.

Linux / файлы и права / ПРОСМОТР и ИЗМЕНЕНИЕ

```
ls -ld <каталог>  
cat <файл>  
sudo -u <пользователь> test -w <каталог>; echo $?  
sudo chmod <режим> <путь>  
sudo chown <владелец>:<группа> <путь>
```

Повторение Linux-блока: ls -d показывает сам каталог, -l права и владельца; cat читает содержимое. test -w проверяет доступ на запись, код 0 означает успех этой проверки, но не гарантирует наличие места. chmod меняет права, chown владельца/группу; они не взаимозаменяемы. Для прохода через каталог нужен x. Не выдавай 777 вместо диагностики.

Linux / inode и поиск / ПРОСМОТР

```
df -i <путь>  
sudo find <учебный_каталог> -xdev -maxdepth 1 \  
-type f -name '<маска>' -print
```

df -i: IUsed/IFree/IUse%, а не свободные байты. find: -xdev не переходит на другую filesystem, -maxdepth ограничивает глубину, -type f выбирает файлы, -name проверяет маску, -print показывает выборку. Кавычки оставляют маску для find, а не shell. Это сначала просмотр; автоматическое удаление по слову cache я не настраивал.

08 / Закрепление

Короткая запись для блокнота

- Сначала воспроизвожу исходный симптом с ограниченным временем ожидания.
- Разделяю name resolution, TCP, TLS, HTTP и прикладной результат.
- Сопоставляю процесс, listener, конфиг и время записи в журнале.
- В проху проверяю два участка, а не только прямой API.
- Исправляю одну подтверждённую причину.
- Повторяю исходный запрос с теми же требованиями безопасности.
- Код 200 без нужного тела или с -k не закрывает соответствующую проблему.
- PASS, показанный в выводе, и PASS, сообщённый словами, фиксирую различно.

Самопроверка

1. Почему работающий listener не исключает timeout?
2. Какие два участка есть у reverse проху и что подтвердил 502 в s10?
3. Почему API 200 не доказывает успех URL проху?
4. Что дополнительно проверяешь, если проху отдаёт старую версию с 200?
5. Чем restart env-зависимого процесса отличается от daemon-reload?
6. Какие поля сертификата сравнишь с URL и текущим временем?
7. Почему переименование файла не меняет SAN, а редактирование SAN ломает подпись?
8. Что остаётся и что теряется при curl -k?

Критерий хорошего ответа

Не просто название команды, а связь: наблюдение -> гипотеза -> различающая проверка -> минимальное изменение -> клиентская приёмка. Не угадывать причину по одному симптому.

Основания и источники

Практика: сообщения и мои скриншоты 24-30 сентября 2026, рабочие отчёты INC-010/011/012 и локальный журнал прогресса для INC-009. При подготовке реферата отчёт INC-009 оставался шаблоном; я опирался на переписку и журнал, не на пустые поля.

Официальные материалы для повторения:

- curl, сертификаты и проверка сервера: <https://curl.se/docs/sslcerts.html>
- curl, параметры клиента: <https://curl.se/docs/manpage.html>
- OpenSSL x509: <https://docs.openssl.org/3.0/man1/openssl-x509/>
- systemd units: <https://www.freedesktop.org/software/systemd/man/latest/systemd.unit.html>

Что ещё не закрыто

Устная защита Session 09. Самостоятельный перенос TLS-проверок на незнакомый кейс. Проверка тела ответа без напоминания. Общий сетевой модуль не объявлен полностью освоенным. Контейнерный блок я выполнил позже 30 сентября; реферат Session 13 оформлен отдельно.